

Closing the Loop on LLM-Generated RTL Assertions with Quality-Aware Formal Verification

Ramesh Krishnamurthy, Danial Chitnis, Themis Prodromakis
School of Engineering, The University of Edinburgh, Edinburgh, United Kingdom
rkrishn3@ed.ac.uk

Abstract—Large language model (LLM) based assertion generation is making formal verification more accessible for Register Transfer Level (RTL) designs, but three practical issues remain. Generated properties can pass for the wrong reason, proof cost can vary widely from one design to another, and failing traces are hard to interpret. This paper presents a lightweight, open-source framework that addresses these issues in one loop. Our method combines mutation-guided refinement to reject weak assertions, including vacuous ones and those that fail to distinguish faulty behavior, a solver-selection stage that chooses among candidate Satisfiability Modulo Theories (SMT) backends using RTL structure, and causal narrative synthesis to explain why a proof failed. Across diverse RTL designs, the framework improves confidence in generated assertions, reduces runtime variability over fixed-solver choices, and produces failure explanations that remain grounded in the counterexample trace. The results suggest that quality-aware closure, rather than assertion generation alone, is the missing step for practical LLM-assisted formal verification.

Index Terms—Formal Verification, LLM, Verification Automation

I. INTRODUCTION

Formal verification (FV) offers a rigorous alternative to simulation for proving hardware correctness, but its deployment remains constrained by the effort needed to write high-quality assertions and configure proof backends effectively. Recent LLM-based frameworks have reduced the front-end burden of assertion generation, including systems that translate specifications into assertions and systems that use both the design specification and the RTL code as input [1]–[3]. More broadly, recent surveys and agentic workflows show growing interest in LLM-assisted hardware design and verification, but they also highlight the need for stronger quality control and evaluation tied to actual proof results from formal tools [4], [5]. Generation alone, however, is not enough. A generated assertion may pass vacuously, run slowly when paired with an unsuitable solver backend, or fail with a trace that is hard to understand.

We target these three bottlenecks directly. First, vacuity remains a known threat in formal verification because properties can hold for trivial reasons rather than because the intended behavior was actually exercised [6]. Second, backend selection remains highly instance-dependent; prior work on SMT formulas and word-level netlists shows that structural features can strongly affect solver performance [7], [8]. Third, raw counterexample traces remain difficult to interpret, especially for users who are new to FV, because long traces often contain many signal changes and the first divergence does

not necessarily coincide with the cycle at which the assertion actually fails.

Recent related work covers important parts of this picture, but not the whole loop. Assertion-generation systems such as AssertLLM, Spec2Assertion, AssertionForge, and DeepAssert improve how properties are synthesized from text, structure, or module context [2], [3], [9], [10]. Recent work has also coupled LLM-generated properties with mutation testing to assess invariant quality [11]. In our flow, mutation serves a different role: mutant outcomes trigger *verification harness refinement* and help decide whether the harness is strong enough to proceed. This distinction is deliberate. Assessing invariant quality is useful as a diagnostic, but our goal is closure: when mutants survive, the framework uses that evidence to strengthen the harness or to recognize that a survivor remains unobservable within the explored proof window. The two strategies are therefore complementary: quality assessment can score a generated property set, whereas mutation-guided refinement uses the same mutation evidence as a control signal for refinement. Agentic workflow systems such as Saarthi show how LLM agents can orchestrate formal tasks [12], but the quality-control layer, i.e., checking assertion meaningfulness, selecting a backend, and explaining failures from trace evidence, remains the gap addressed here.

This paper presents a quality-aware framework for LLM-generated RTL assertions. The framework combines: (i) *Mutation-Guided Refinement (MGR)* to reject weak assertions, including vacuous ones and those that fail to distinguish faulty behavior, (ii) an *architecture-aware solver selector* that uses RTL structure before the design is translated into solver-specific forms, and (iii) *Causal Narrative Synthesis (CNS)* to summarize failing traces in human-readable form. The contribution is a compact loop that improves confidence in generated assertions, stabilizes runtime, and makes failures easier to debug in an *open-source* flow built on Yosys-based tooling [13].¹

II. FRAMEWORK

Figure 1 summarizes the proposed flow. Given an RTL design and a natural-language specification, an LLM first produces an initial verification harness. In this paper, a verification harness means the formal assertions, assumptions, and any

¹Code and benchmark artifacts are available at github.com/EEEdigits.

helper logic used to verify the RTL design; we refer to it hereafter simply as the *harness*. The framework then profiles RTL structure once, selects a solver backend, and emits the proof configuration used in subsequent audit runs. MGR injects synthetic RTL faults and checks whether the current assertions detect them under that fixed configuration. Surviving mutants trigger harness refinement. Once the assertion set kills the relevant mutants at an acceptable rate, the final proof outcome is recorded; if the proof fails, the resulting counterexample is translated into a short root-cause narrative. The front-end harness stage is included for completeness, but the technical contributions begin after that point: MGR, solver selection, and CNS. In the present experiments, the initial harness is generated with RTL context because the study starts from existing public RTL designs and treats harness generation as a bootstrap step rather than a contribution under evaluation. Most of the quality-control loop is tool-driven: the mutation audit and kill-rate computation in MGR, together with RTL profiling and solver selection, are *deterministic* stages, while LLM calls are used only for harness construction/refinement and for natural-language failure summaries.

A. Mutation-Guided Refinement

The goal of MGR is to *test the test* [14]. Starting from the original RTL D and the current assertion set $\mathcal{A}$, the framework injects faults into D and checks whether $\mathcal{A}$ still passes. Any passing mutant reveals a weakness in the harness; the survived mutant is then fed back to the LLM for the next refinement round. Vacuity is one failure mode of weak assertions, but MGR addresses a broader problem: whether the current harness is sufficiently discriminative under injected faulty behaviors. Satisfiability checks may be used to detect strict vacuity, while MGR strengthens properties that still pass but fail to distinguish relevant mutants.

Unlike prior mutation-based quality checks for generated properties [10], [11], MGR uses mutant outcomes to decide whether the harness should be accepted, refined again, or interpreted (cautiously) when a surviving mutant does not produce an observable failure within the explored proof window. For a mutant set M , we define the effective kill rate as:

$$\rho_{\text{eff}} = \frac{\# \text{Killed}}{\# \text{Total} - \# \text{Invalid}}. \quad (1)$$

In the current workflow, we set the refinement trigger to $T = 85\%$: if the effective kill rate falls below this value, another refinement round is started. This threshold is a practical floor rather than a final quality target; in our experiments, robust harnesses typically converged to 90–100% effective kill on non-invalid mutants. If the effective kill rate remains low, surviving mutants are fed back to the LLM for strengthening, turning formal verification into an *adversarial refinement loop* rather than a one-shot generation task.

MGR workflow: Each mutant is verified independently under the same harness and proof configuration. Results are classified as KILLED (assertions detect the mutant), SURVIVED (missed by the harness), INVALID (the edit no

longer produces a meaningful hardware variant), or TIME-OUT/INCONCLUSIVE. These per-mutant outcomes and aggregate audit results form the audit evidence returned to the refinement stage. Surviving mutants are fed back to the LLM to strengthen the next harness round. INVALID mutants are excluded from the denominator of ρ_{eff} because they do not exercise the harness.

B. Architecture-Aware Solver Selection

To reduce runtime unpredictability, we profile each design before translation into solver-specific forms. We compute a lightweight structural score:

$$\chi = (D_A \times W) + D_M + \frac{D_R}{I_C}, \quad (2)$$

where W is dominant datapath width, D_A is arithmetic density, D_M is mux density, D_R is total memory size in bits, and I_C is index complexity. The terms are normalized to a common reference scale and combined as a weighted sum for the final solver decision, preventing the memory term from dominating on small designs. Intuitively, high D_M indicates control-steering logic, high $D_A \times W$ indicates wide datapaths, and high D_R/I_C indicates memory-sensitive behavior. Unlike prior selectors that operate on SMT formulas or lowered Btor2 tasks [7], [8], our selector works directly on RTL structure. This lets the flow choose a backend *before* building the full proof instance, using signals that hardware engineers can understand and sanity-check.

C. Causal Narrative Synthesis

When verification fails, the framework extracts the failing property, a compact slice of the trace, and the most relevant signal transitions, then prompts the LLM to produce a concise explanation of the trigger condition and violated expectation. Counterexample explanation is not a new problem in formal verification; prior work has studied causality-based methods for highlighting why a trace violates a specification [15], and industrial studies show that explanation quality directly affects whether engineers can make effective use of formal verification results [16]. Our focus here is different: CNS is evaluated as a *faithfulness problem* rather than a fluency problem, so the explanation should point to the right cycle, the right signals, and the right failure mechanism without introducing claims that are not supported by the trace. This makes CNS a measurable *debugging aid* rather than a presentation-only add-on. In the current workflow, the model is guided by the failing step and trace-based failure cues so that the explanation remains tied to the actual counterexample.

III. EXPERIMENTAL EVALUATION

A. Setup

The framework is implemented on top of a white-box Yosys/SymbiYosys flow [13]. An orchestration layer extracts structural metrics, selects a solver, and emits the SymbiYosys configuration automatically. In our current artifact, initial harness generation uses GPT 5.4 and CNS uses Claude Sonnet 4.6 (Anthropic). Each MGR campaign uses 20 mutants per

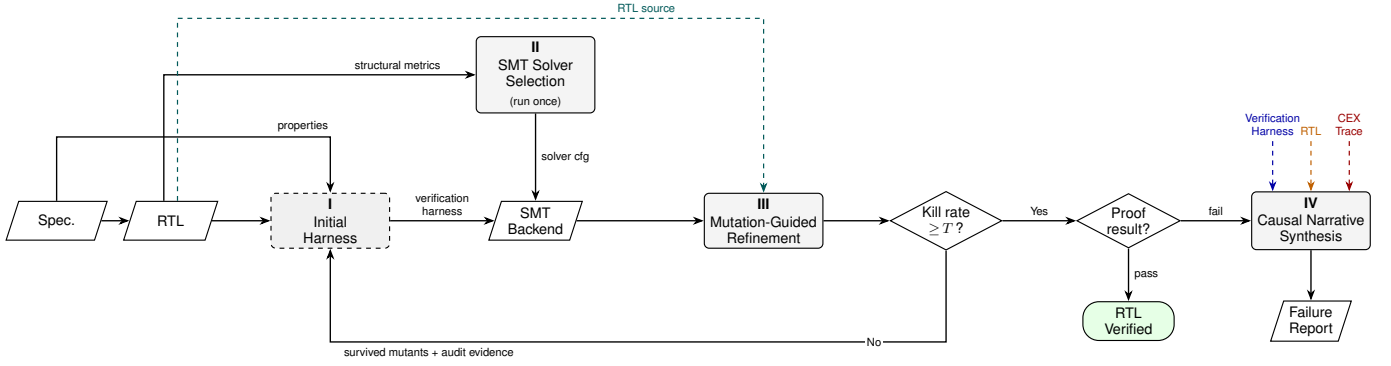


Fig. 1. The proposed quality-aware verification loop. The initial harness stage is shown only as a bootstrap. Solver selection is computed once from RTL structure and reused during mutation-guided refinement. The paper’s technical contributions begin with solver selection and continue through mutation-guided refinement and concise failure narratives.

TABLE I
BENCHMARK CLASSES AND VERIFICATION DEPTHS.

Class	Design	k
Protocol	I2C Master	80
Protocol	UART (Full)	120
Memory	Sync. FIFO	15
Memory	SDRAM Ctrl	500
Data	FFT 256	80
Data	SHA3	80
Peripheral	VGA LCD	80
Control	uP8 (ADD/ISA)	45

round. We evaluate MGR on seven RTL designs: UART_RX, UART_Full, UP8, SHA3, SDRAM, VGA, and FFT. Fig. 2 shows three supplementary learning curves: UART_TX, I2C, and VGA. For solver selection, we use Yices, Bitwuzla, Z3, and CVC5 as representative SMT backends [17]–[20]. We profile each design and report average step latency (ASL),

$$\text{ASL} = \frac{T_{\text{total}}}{k}, \quad (3)$$

where T_{total} is wall-clock proof runtime and k is the bounded depth [21]. Table I lists the benchmark classes and chosen depths.

B. Mutation-Guided Refinement Results

Table II summarizes the best achieved MGR round across seven completed campaigns. The overall pattern is strong and consistent: six designs finish between 90% and 100% effective kill rate. UP8 and SHA3 reach 100% in their first round. UART_RX, UART_Full, and VGA stop at 90%, and FFT reaches 93.8%. SDRAM is the only clear outlier. Its final score remains lower because some mutations change timing-related internal behavior, but within the bounded number of cycles (explored by the proof) they do not produce any observable failure, so the mutant survives even though the harness may not actually be weak. In other words, additional refinement stopped yielding new kills. We refer to that point as a *plateau*.

Across the suite, the most important cross-design pattern is the emergence of source-level edits that change the RTL text but not what the formal tool can observe. We refer to these as *true equivalents*. Three categories recur repeatedly: delay annotations that disappear in RTL-level checking, inactive

TABLE II
BEST MGR ROUND PER DESIGN (‘R’ = ROUND).

Design	Best	Eff. Kill	Observation
UART_RX	R1	90.0%	delay-only survivors
UART_Full	R1	90.0%	delay-only survivors
UP8	R1	100.0%	instruction-level checks
SHA3	R1	100.0%	fully specified datapath behavior
SDRAM	R4	36.8%	no new kills after timing edits
VGA	R2	90.0%	one-round refinement gain
FFT	R2	93.8%	one residual inactive-branch edit

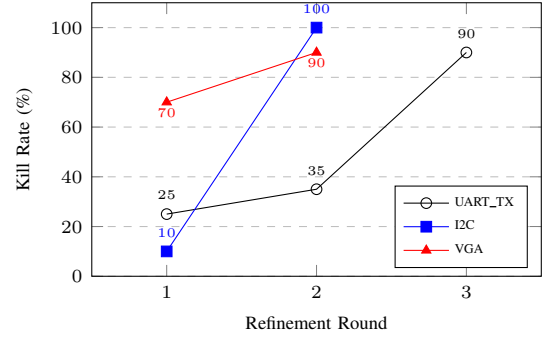


Fig. 2. Illustrative MGR refinement trajectories for UART_TX, I2C, and VGA; Table II gives the cross-design summary.

preprocessor branches, and timing-setting edits whose effect does not appear within the number of cycles explored by the proof. This distinction matters because a low plateau is not automatically evidence of a weak harness. In practice, MGR is most useful when it shows which survivors come from weak assertions and which come from mutations that the formal setup cannot observe.

C. Solver Selection Results

Table III reports solver performance across the current benchmark suite. The pattern is consistent with the proposed selector: control-heavy designs with small widths, such as I2C, VGA, and uP8, favor Yices, while the wide datapath benchmark SHA3 favors Bitwuzla. The table also shows why backend choice cannot be treated as a *default setting*. Cases like FFT256, SHA3, and VGA, timeout under Z3 despite completing on other backends. The selector is therefore not chasing only small runtime gains; it is avoiding bad choices

TABLE III

NORMALIZED SOLVER PERFORMANCE (ASL IN S/STEP); T/O = TIMEOUT.

Benchmark	Bitwuzla	CVC5	Z3	Yices
I2C Master	0.125	5.913	0.713	0.038
UART (Full)	0.008	T/O	0.017	0.008
Sync. FIFO	2.733	T/O	27.733	6.800
SDRAM Ctrl	0.026	T/O	0.290	0.024
FFT 256	1.338	15.975	T/O	0.613
SHA3	0.250	9.413	T/O	0.500
VGA LCD	0.388	8.725	T/O	0.150
uP8 (ADD)	3.556	44.000	0.156	0.044
uP8 (ISA)	0.667	T/O	18.756	0.556

TABLE IV

SOLVER-SELECTION BASELINE SUMMARY ON THE CURRENT SUITE.

Policy	Mean ASL	Relative to Yices
Always Bitwuzla	1.010	1.04×
Always Yices	0.970	1.00×
Rule-based selector	0.491	0.51×
Oracle best	0.491	0.51×

that turn a solvable proof into a timeout. The memory-heavy uP8(ISA) case provides a useful calibration point: although its structure suggests an array-friendly backend (like Z3), Yices still wins at the tested proof bound. This shows that memory density alone is not enough to choose the solver, and that the selector must capture such *hardware-specific tradeoffs*.

To make the selector easier to interpret, Table IV compares our rule-based policy against fixed-solver baselines. “Always Yices” and “Always Bitwuzla” mean that the same solver is used for every benchmark. “Oracle” is the unattainable upper bound that always picks the fastest solver after seeing the result. On the current benchmark suite, the rule-based selector matches the oracle solver choice on every benchmark and reduces mean ASL by roughly 49% compared with always using Yices.

D. Causal Narrative Results

We evaluated CNS on eight failed counterexamples drawn from four designs (FFT, VGA, UART_RX, and SHA3). For each case, the ground truth consists of the manually established failing cycle, key signals, and failure mechanism. This ground truth was written before any CNS run and then verified in a second blind evaluation pass. Table V compares three settings: R1 baseline is the original zero-shot prompt, R2 refined is the shared improved prompt evaluated on six cases without case-specific hints, and R2 + hints adds such hints for the two hardest cases. The four binary scoring criteria answer the four basic debug questions: *when* (correct failing cycle), *where* (correct key signal), *what kind* (correct mechanism class), and *honest* (zero unsupported signal claims). A response is *faithful* only when all four hold.

Even in the baseline, CNS identifies the correct signals in all eight cases and never invents signal names. Its errors are mainly about *when* and *how*: in longer traces, the baseline sometimes reports the first cycle where signals begin to diverge rather than the step at which the assertion fails, and it sometimes assigns a broad label such as “timing error”

TABLE V

CNS FAITHFULNESS ACROSS PROMPT SETTINGS.

Metric	R1 baseline	R2 refined	R2 + hints
Correct cycle	4/8	6/6	8/8
Correct signal	8/8	6/6	8/8
Correct mechanism	5/8	6/6	8/8
Unsupported claims	0	0	0
Overall faithful	3/8	$\geq 6/8$	8/8

when the trace supports a more specific class. The refined prompt addresses both issues by stating the assertion-failure step explicitly and by giving clearer descriptions for each failure class. All six cases evaluated without case-specific hints are then faithful. The remaining two require extra hints, so the conservative claim is that refined CNS reaches at least 75% faithfulness with the shared prompt, rising to 100% with additional hints. Across all settings, signal identification is correct and no unsupported signal names are introduced. After refinement, CNS explanations remain short while preserving the key debug facts: responsible signal, failure type, and why the assertion fires.

IV. DISCUSSION AND CONCLUSION

The results suggest that the central problem in LLM-assisted FV is not property generation alone, but *quality-aware closure*. MGR is the strongest component: it directly addresses false confidence from weak properties, and six of seven designs reach 90–100% effective kill rate. The SDRAM plateau illustrates an important structural limit: some timing-oriented mutations do not produce an observable failure within feasible proof depths, so they cannot be killed regardless of harness quality. Recognizing that limit is itself a useful outcome of MGR. The solver selector provides a lightweight hardware-level rule for choosing the proof backend. CNS demonstrates that a zero-shot LLM reliably identifies the correct signals without hallucination, and a targeted two-part prompt refinement lifts overall faithfulness from 37% to at least 75%. Coverage remains an important metric in AI-for-FV research, but this paper focuses on a narrower and more defensible notion of quality: adversarial coverage through MGR, runtime stability through solver selection, and trace faithfulness through CNS. Taken together, the results support a simple message: LLM-generated assertions become materially more *trustworthy* when they are mutation-tested, structurally profiled, and paired with an open verification stack. Future work will add explicit coverage-closure metrics, compare the structural score (χ) against additional solver-selection baselines, and extend the CNS evaluation to counterexamples from live regressions rather than mutation-triggered failures alone.

ACKNOWLEDGMENT

The authors acknowledge UK Research and Innovation (UKRI) via its Engineering and Physical Sciences Research Council (EPSRC) funding for the AI Hub for Productive Research and Innovation in Electronics (APRIL), grant number EP/Y029763/1. Also, EDINA and ISG@University of Edinburgh for AI services.

REFERENCES

- [1] M. Li, W. Fang, Q. Zhang, and Z. Xie, “Spec1lm: Exploring generation and review of VLSI design specification with large language model,” *CoRR*, vol. abs/2401.13266, 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2401.13266>
- [2] Z. Yan, W. Fang, M. Li, M. Li, S. Liu, Z. Xie, and H. Zhang, “Assert1lm: Generating hardware verification assertions from design specifications via multi-llms,” in *Proceedings of the 30th Asia and South Pacific Design Automation Conference, ASPDAC 2025, Tokyo, Japan, January 20-23, 2025*, Y. Nakamura and Y. Wang, Eds. ACM, 2025, pp. 614–621. [Online]. Available: <https://doi.org/10.1145/3658617.3697756>
- [3] Y. Bai, G. B. Hamad, S. Suhaib, and H. Ren, “Assertionforge: Enhancing formal verification assertion generation with structured representation of specifications and RTL,” *CoRR*, vol. abs/2503.19174, 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2503.19174>
- [4] M. Abdollahi, M. Dargahi, M. Rashidian, M. Shalan, and R. Karri, “Hardware design and verification with large language models: A literature survey, challenges, and open issues,” *Electronics*, vol. 14, no. 2, p. 265, 2025.
- [5] D. N. Gadde, K. K. Radhakrishna, V. N. Viswambharan, A. Kumar, D. Lettnin, W. Kunz, and S. Simon, “Hey AI, generate me a hardware code! agentic AI-based hardware design & verification,” in *Proceedings of the 38th SBC/SBMicro/IEEE Symposium on Integrated Circuits and Systems Design (SBCCI)*, 2025.
- [6] I. Beer, S. Ben-David, C. Eisner, and Y. Rodeh, “Efficient detection of vacuity in temporal model checking,” in *Proceedings of the 12th International Conference on Computer Aided Verification (CAV)*, ser. LNCS, vol. 1855. Springer, 2001.
- [7] J. Scott, A. Niemetz, M. Preiner, S. Nejati, and V. Ganesh, “Machsmt: A machine learning-based algorithm selector for SMT solvers,” in *Tools and Algorithms for the Construction and Analysis of Systems*, ser. Lecture Notes in Computer Science, J. F. Groote and K. G. Larsen, Eds., vol. 12652. Springer, 2021, pp. 303–325.
- [8] Z. Lu, P. Chien, N. Lee, A. Gurfinkel, and V. Ganesh, “Btor2-select: Machine learning based algorithm selection for hardware model checking,” in *Computer Aided Verification*, ser. Lecture Notes in Computer Science, vol. 15931. Springer, 2025.
- [9] F. Wu, E. Pan, R. Kande, M. Quinn, A. Tyagi, D. K. Houngrinou, J. Rajendran, and J. Hu, “Spec2assertion: Automatic pre-RTL assertion generation by LLMs with progressive regularization,” *CoRR*, vol. abs/2505.07995, 2025.
- [10] Y. Wang, J. Zhou, H. Lyu, Z. Chao, T. Wang, and H. Li, “Deepassert: An LLM-aided verification framework with fine-grained assertion generation for modules with extracted module specifications,” *CoRR*, vol. abs/2509.14668, 2025.
- [11] M. Hassan, S. Ahmadi-Pour, K. Qayyum, C. K. Jha, and R. Drechsler, “LLM-guided formal verification coupled with mutation testing,” in *Proceedings of the Design, Automation and Test in Europe Conference (DATE)*, 2024, pp. 1–2.
- [12] A. Kumar, D. N. Gadde, K. K. Radhakrishna, and D. Lettnin, “Saarthi: The first AI formal verification engineer,” *CoRR*, vol. abs/2409.06766, 2024.
- [13] C. Wolf, J. Glaser, and J. Kepler, “Yosys-a free verilog synthesis suite,” in *Proceedings of the 21st Austrian Workshop on Microelectronics (Austrochip)*, vol. 97, 2013, pp. 1–6.
- [14] Y. Jia and M. Harman, “An analysis and survey of the development of mutation testing,” *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.
- [15] I. Beer, S. Ben-David, H. Chockler, A. Orni, and R. Treffer, “Explaining counterexamples using causality,” in *Computer Aided Verification*, ser. Lecture Notes in Computer Science, vol. 6174. Springer, 2010, pp. 94–108.
- [16] A. P. Kaleeswaran, A. Nordmann, T. Vogel, and L. Grunske, “A user study for evaluation of formal verification results and their explanation at bosch,” *Empirical Software Engineering*, vol. 28, no. 125, 2023.
- [17] B. Dutertre, “Yices 2.2,” in *Computer Aided Verification - 26th International Conference, CAV 2014, Held as Part of the Vienna Summer of Logic, VSL 2014, Vienna, Austria, July 18–22, 2014. Proceedings*, ser. Lecture Notes in Computer Science, A. Biere and R. Bloem, Eds., vol. 8559. Springer, 2014, pp. 737–744.
- [18] A. Niemetz and M. Preiner, “Bitwuzla,” in *Computer Aided Verification - 35th International Conference, CAV 2023, Paris, France, July 17–22, 2023. Proceedings, Part II*, ser. Lecture Notes in Computer Science, C. Enea and A. Lal, Eds., vol. 13965. Springer, 2023, pp. 3–17.
- [19] L. de Moura and N. Bjørner, “Z3: An efficient SMT solver,” in *Tools and Algorithms for the Construction and Analysis of Systems, TACAS 2008*, ser. Lecture Notes in Computer Science, C. R. Ramakrishnan and J. Rehof, Eds., vol. 4963. Springer, 2008, pp. 337–340.
- [20] H. Barbosa, C. W. Barrett, M. Brain, G. Kremer, H. Lachnitt, M. Mann, A. Mohamed, M. Mohamed, A. Niemetz, A. Nötzli, A. Ozdemir, M. Preiner, A. Reynolds, Y. Sheng, C. Tinelli, and Y. Zohar, “cvc5: A versatile and industrial-strength SMT solver,” in *Tools and Algorithms for the Construction and Analysis of Systems - 28th International Conference, TACAS 2022, Held as Part of ETAPS 2022, Munich, Germany, April 2–7, 2022. Proceedings, Part I*, ser. Lecture Notes in Computer Science, D. Fisman and G. Roşu, Eds., vol. 13243. Springer, 2022, pp. 415–442.
- [21] A. Biere, A. Cimatti, E. M. Clarke, and Y. Zhu, “Symbolic model checking without BDDs,” in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, ser. Lecture Notes in Computer Science, vol. 1579. Springer, 1999, pp. 193–207.